pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Treasury Yields

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The problem: a reference sheet from real data

The data: real published figures, for the first time in the course

2,940
rows, one per date

14
tenors, 1 Mo to 30 Yr

2015-01-02
to 2026-10-02

The first two rows and the last row, five of the 14 tenors, in percent

date	1 Mo	2 Yr	10 Yr	30 Yr
2015-01-02	0.02	0.66	2.12	2.69
2015-01-05	0.02	0.68	2.04	2.60
2026-10-02	4.04	4.83	5.28	5.63

```
raw = pd.read_csv(data_folder + 'treasury_par_yields.csv')
print(raw.shape)   # (2940, 15): the date and 14 tenors
```

The desk asks for

Load the file as a time series

Check its span, absent days, and blanks

Describe yields by tenor and by year

Describe how they moved, day by day

The sheet is a description. It says what the yields were and when. It does not say why they moved, or where they are going.

These are par yields on Treasury securities, as published. They are not part of the invented portfolio, and no bond from the portfolio appears today.

Source: U.S. Department of the Treasury, Daily Treasury Par Yield Curve Rates. Retrieved 2026-10-03. Figures as published.

https://home.treasury.gov/resource-center/data-chart-center/interest-rates/TextView?type=daily_treasury_yield_curve

A date as the index

A plain read_csv

date is a column of text: str

the row labels are the numbers 0 to 2,939

raw.index.year raises an AttributeError

With parse_dates and index_col

date becomes the row labels: a DatetimeIndex

2,940 rows and 14 columns, one per tenor

yields.index.year works, with no new column

```
yields = pd.read_csv(file, parse_dates=['date'], index_col='date')   # dates as dates, and as the row labels
```

Select	pandas	Returns	Excel
One cell	`yields.loc['2020-03-09', '10 Yr']`	0.54 percent	`=XLOOKUP(DATE(2020, 3, 9), A2:A2941, M2:M2941)`
A year	`yields.loc['2020']`	251 rows, not 366	`Data, Filter, tick 2020 on column A`
A slice	`yields.loc['2019-12-30':'2020-01-03']`	4 rows, both ends kept	`Data, Filter, Date Filters, Between`

yields.loc['2020-03'] returns the 22 rows of March 2020. The file has no row for 2020-01-01, so the slice returns 4 rows.

In Excel `=ISNUMBER(A2)` TRUE once opened `=DATEVALUE(A2)` for a date still held as text `=YEAR(A2)` returns 2015

The sheet: date in column A, tenors in columns B to O, rows 2 to 2941. The 2 Yr is column I and the 10 Yr is column M.

What is missing, and why it matters

126

weekdays with no row

190

rows in 2026, a part year

5,432

blank cells, in three tenors

Absent is not blank

The file has no Saturday or Sunday, and 126 of the 3,066 weekdays in its span have no row at all: 2024-07-04 is one. isna() cannot find a row that is not there. Build the dates that could be there, and compare.

```
weekdays = pd.date_range(yields.index.min(),
                         yields.index.max(), freq='B')
no_row = weekdays.difference(yields.index)   # 126 dates
yields.isna().sum()                  # blanks by column
yields['2 Mo'].first_valid_index()   # 2018-10-16
```

`=NETWORKDAYS(A2, A2941) - COUNT(A2:A2941)` returns 126

`=SUMPRODUCT(--(WEEKDAY(A2:A2941, 2) > 5))` returns 0

Three tenors begin partway through the file

Tenor	First value	Values	Blanks
2 Mo	2018-10-16	1,991	949
4 Mo	2022-10-19	989	1,951
1.5 Month	2025-02-18	408	2,532

Blank before its first value, complete after. Nothing is filled.

What a late start does to an average: 2018

Column and days	Values	Mean, percent
2 Mo, from 2018-10-16	51	2.33
3 Mo, all of 2018	249	1.97
3 Mo, the same 51 days	51	2.38

```
yields.loc['2018', '2 Mo'].count()   # 51 of 249 rows
```

`=COUNTBLANK(D2:D2941)` returns 949. =COUNT(D2:D2941) returns 1,991

`=AVERAGE(D2:D2941)` leaves empty cells out, as mean() does

Wide against long

Wide: one row per date, one column per tenor

date	1 Mo	3 Mo	10 Yr
2015-01-02	0.02	0.02	2.12
2015-01-05	0.02	0.03	2.04
2015-01-06	0.02	0.03	1.97

`melt()` →

← `pivot_table()`

Long: one row per date and tenor

date	tenor	yield_pct
2015-01-02	1 Mo	0.02
2015-01-05	1 Mo	0.02
2015-01-06	1 Mo	0.02

Wide suits a time series: a tenor is a column, and a row is the curve on one date. Long lets the tenor be grouped by.

41,160

2,940 dates times 14 tenors

5,432

missing yields, kept by melt()

35,728

rows if blanks were dropped

```
long = yields.reset_index().melt(id_vars='date',
    var_name='tenor', value_name='yield_pct')
long['year'] = long['date'].dt.year
year_tenor = pd.pivot_table(long, values='yield_pct',
    index='year', columns='tenor', aggfunc='mean')
year_tenor = year_tenor[tenors]   # back in tenor order
```

The same in Excel

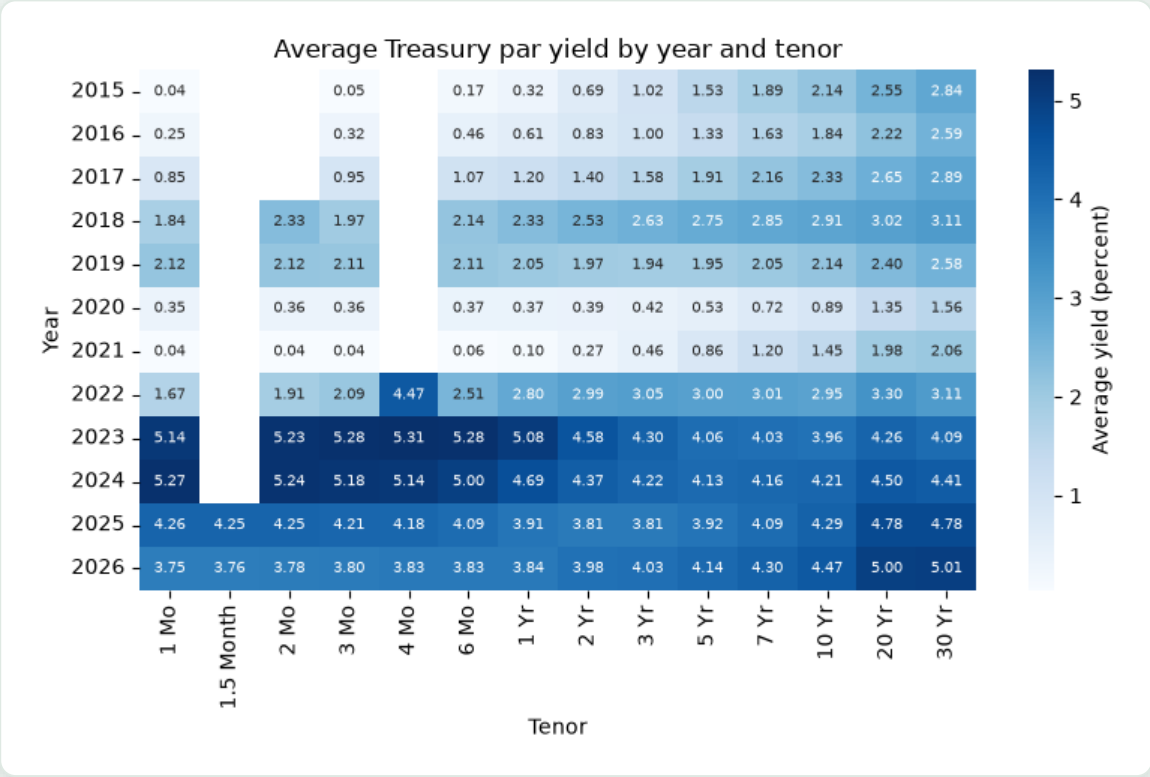
Wide to long is called unpivoting. Data, From Table/Range opens Power Query. Select the date column, then Transform, Unpivot Columns, Unpivot Other Columns.

Power Query leaves out the empty cells when it unpivots. melt() keeps them as missing values.

The table by year and tenor is then a pivot table: date grouped by Years in Rows, the tenor in Columns, and Average of the value in Values.

No number changed: the same 41,160 cells, rearranged.

By tenor and by year



The 10 Yr in five of the twelve years, in percent

10 Yr by year	Mean	Low	High	Range
2017	2.33	2.05	2.62	57 bp
2020	0.89	0.52	1.88	136 bp
2022	2.95	1.63	4.25	262 bp
2025	4.29	3.97	4.79	82 bp
2026, to 2 October	4.47	3.97	5.29	132 bp

Widest range: 262 bp in 2022. Narrowest: 57 in 2017. In 2023 and 2024 the 3 Mo mean was above the 10 Yr mean. The dark 4 Mo cell for 2022, 4.47, averages only the 50 dates from 2022-10-19.

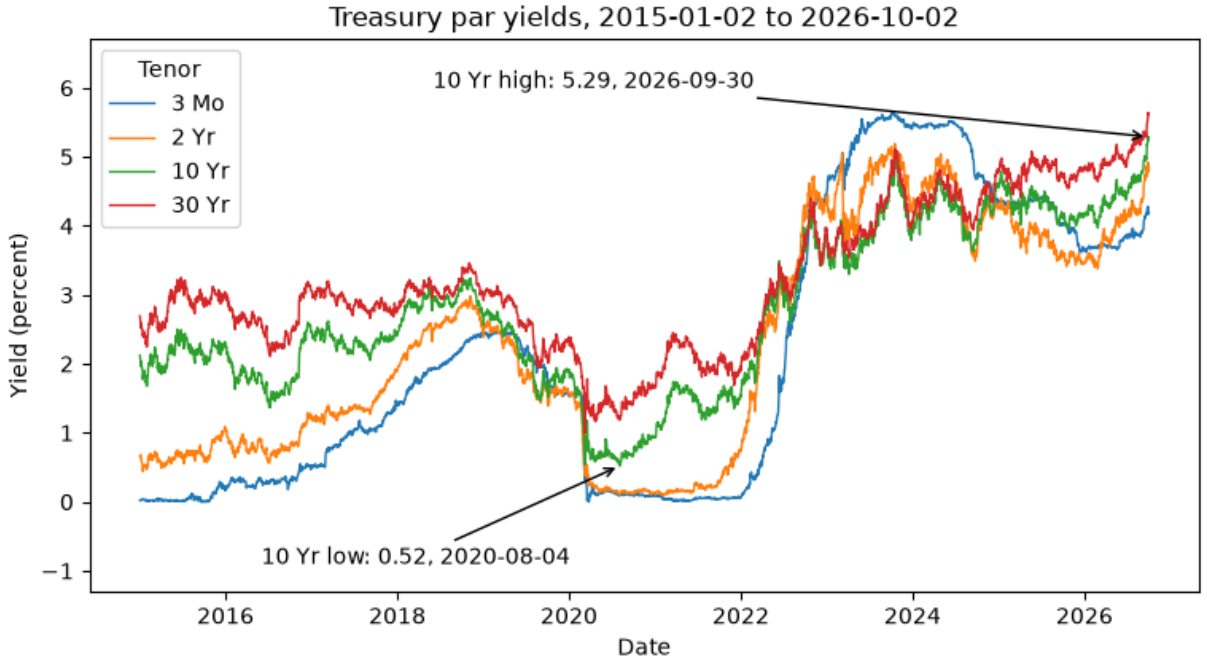
`=AVERAGEIFS(M2:M2941, P2:P2941, 2020)` returns 0.89

`=MAXIFS(M2:M2941, P2:P2941, 2020)` returns 1.88

P holds =YEAR(A2). Heatmap: Conditional Formatting, Color Scales.

```
by_year = yields.groupby(yields.index.year)['10 Yr'].agg(['mean', 'min', 'max'])
sns.heatmap(year_tenor.round(2), annot=True, fmt='.2f', cmap='Blues', ax=ax)
```

Yields over time




```
ax.plot(yields.index, yields[tenor], linewidth=1, label=tenor)
yields['10 Yr'].idxmax()   # the date of the highest value
month_end = yields.resample('ME').last()   # (142, 14)
```

Lowest and highest, in percent, with dates

Tenor	Lowest	On	Highest	On
3 Mo	0.00	2015-09-18	5.63	2023-10-06
2 Yr	0.09	2021-02-05	5.19	2023-10-17
10 Yr	0.52	2020-08-04	5.29	2026-09-30
30 Yr	0.99	2020-03-09	5.64	2026-09-30

The 3 Mo dates are the first of ties: 13 days at 0.00 and 2 days at 5.63. idxmax() and idxmin() return the first date.

From daily to month end

2,940 daily rows become 142 monthly rows. The row labelled 2015-01-31 holds 1.68, the 10 Yr on 2015-01-30. The mean of that month is 1.88. The last row, 2026-10-31, holds the values of 2026-10-02: a part month.

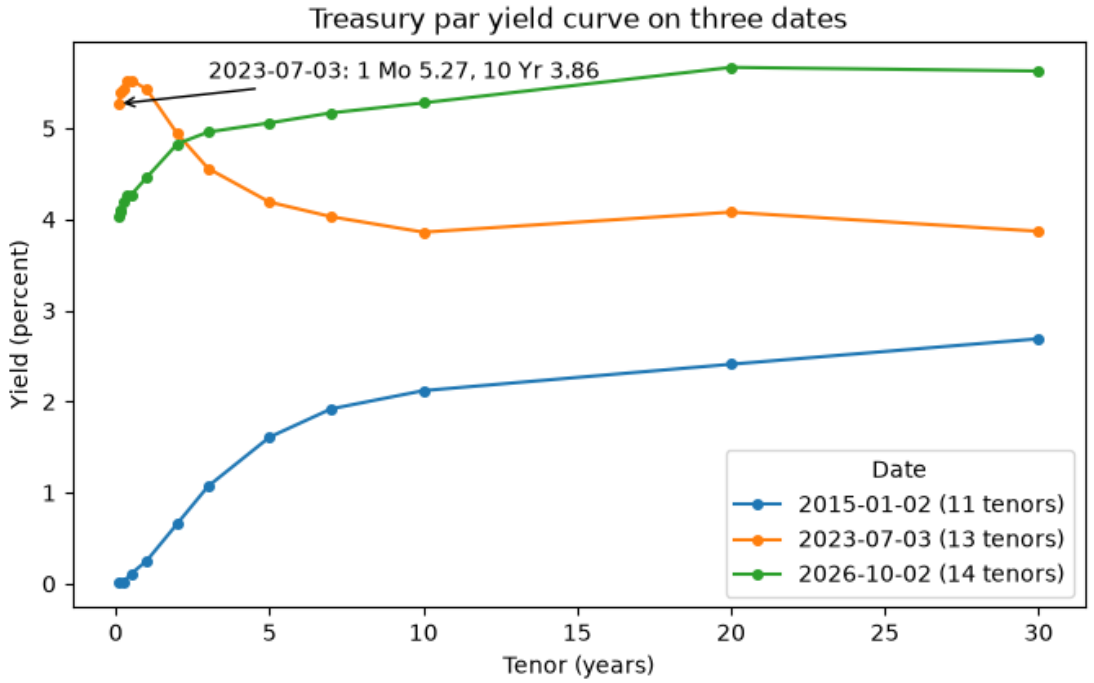
The same in Excel

`=XLOOKUP(MAX(M2:M2941), M2:M2941, A2:A2941)` 2026-09-30

`=XLOOKUP(DATE(2015, 1, 31), A2:A2941, M2:M2941, , -1)` 1.68

Chart: column A and the four tenors, then Insert, Line.

The curve on chosen dates




```
curve = yields.loc[day, tenors].dropna()
x = [tenor_years[tenor] for tenor in curve.index]
ax.plot(x, curve.values, marker='o', markersize=4)
```

Three dates chosen by a rule, yields in percent

Date	Rule	1 Mo	2 Yr	10 Yr	30 Yr	Points
2015-01-02	first date	0.02	0.66	2.12	2.69	11
2023-07-03	lowest spread	5.27	4.94	3.86	3.87	13
2026-10-02	last date	4.04	4.83	5.28	5.63	14

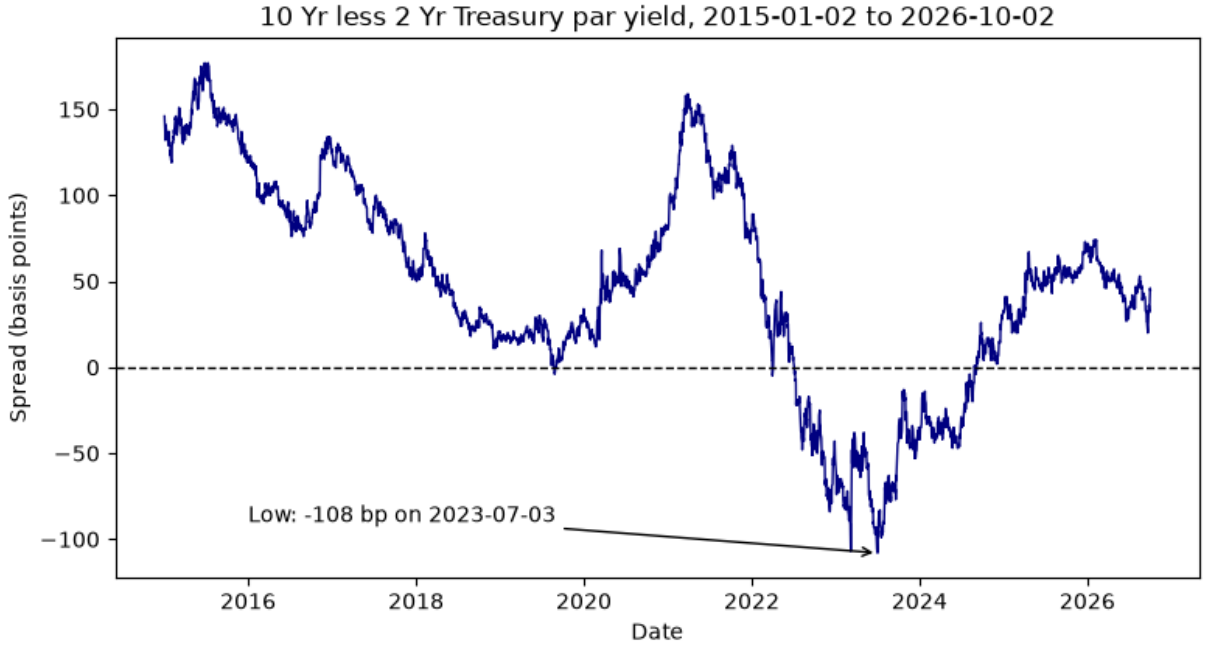
2023-07-03 is the date the 10 Yr was furthest below the 2 Yr. On it the 1 Mo, 5.27, was 141 basis points above the 10 Yr, 3.86. On 2026-10-02 the 20 Yr, 5.67, was above the 30 Yr. The points differ because three tenors begin partway: dropna() draws the line through the tenors that exist.

The x axis is in years, so the six tenors up to 6 Mo sit close together. That is the true spacing.

The same in Excel

Tenor lengths in one row, 0.083 for 1 Mo up to 30 for 30 Yr, the yields beneath. Insert, Scatter, Scatter with Straight Lines and Markers places each point by its x value. A Line chart spaces text categories evenly.

A spread between two tenors




```
yields['spread_bp'] = ((yields['10 Yr'] - yields['2 Yr'])
                       * 100).round().astype(int)
ax.axhline(0, color='black', linewidth=1, linestyle='--')
below = yields['spread_bp'] < 0   # below.sum() is 544
```

10 Yr less 2 Yr, in basis points, on 2,940 days

48.1
mean over the span

177
high, first on 2015-06-26

-108
the low, set on 2023-07-03

Year	2019	2022	2023	2024	Total
Days below zero	3	125	250	166	544

Below the dashed zero line, the 2 Yr was above the 10 Yr. That was 544 of 2,940 days, 18.5 percent, the first on 2019-08-27 and the last on 2024-09-05, and every date in the file in 2023. On 7 more days the spread was exactly zero: < gives 544, <= gives 551.

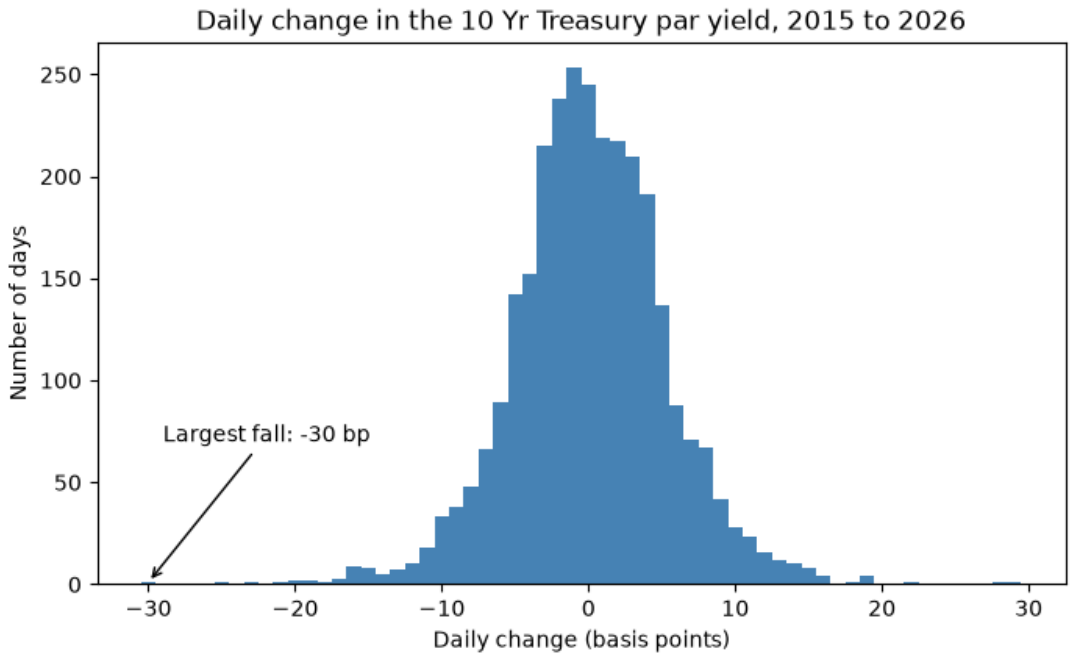
The same in Excel, with the year in column P

`=(M2 - I2) * 100` in column Q: 146 in the first row

`=COUNTIF(Q2:Q2941, "<0")` returns 544

`=COUNTIFS(P2:P2941, 2023, Q2:Q2941, "<0")` returns 250

Daily changes




```
changes = (yields[main].diff() * 100).round()
changes['10 Yr'].count()       # 2939: the first is NaN
changes['10 Yr'].nlargest(5)   # values with their dates
```

The largest one-day moves in the 10 Yr, in basis points

Date	Rise
2020-03-17	29
2022-06-13	28
2020-03-10	22
2016-11-09	19
2022-09-22	19

Date	Fall
2022-11-10	-30
2022-09-28	-25
2023-03-10	-23
2020-03-16	-21
2020-03-09	-20

A rise of 19 happened on four dates, and nlargest(5) keeps the earliest two. Four of the ten dates are in March 2020 and four are in 2022. The file does not say why.

2,939 changes: 1,351 up, 1,343 down, 245 unchanged. Standard deviation 5.30 bp. 75.5 percent were within 5 bp of zero.

The same in Excel

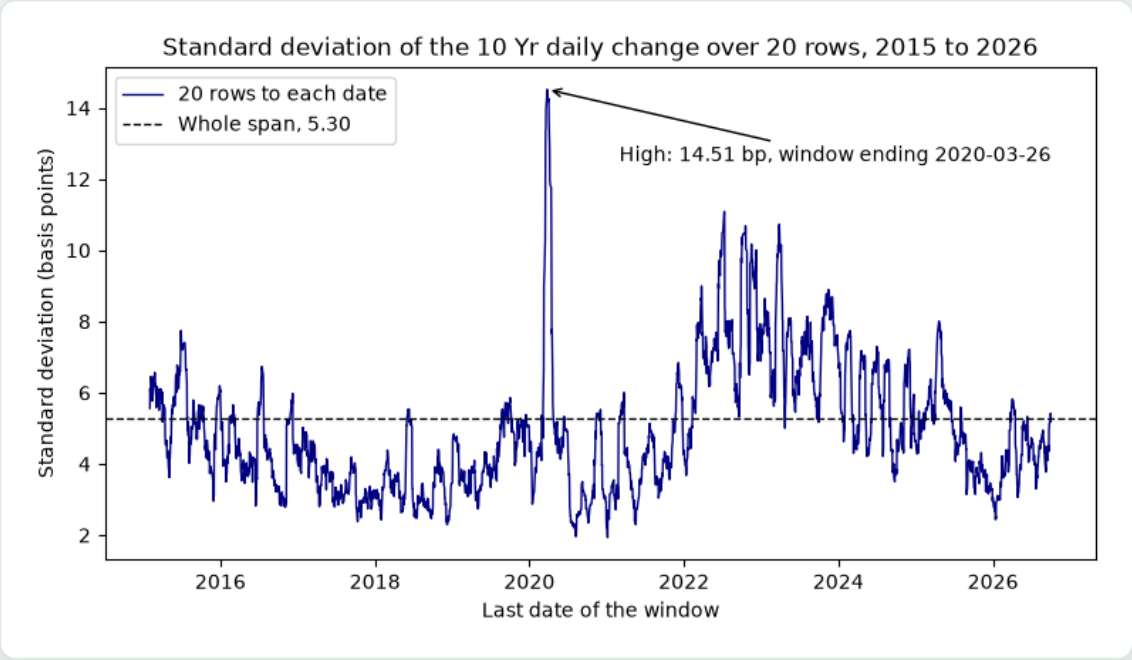
`=(M3 - M2) * 100` in R3, filled down. Gives -8 on 2015-01-05

`=XLOOKUP(MAX(R3:R2941), R3:R2941, A3:A2941)` 2020-03-17

`=LARGE(R3:R2941, 2)` returns 28. =SMALL(R3:R2941, 2) returns -25

`=STDEV.S(R3:R2941)` returns 5.30

A moving window: the 20 rows ending on each date




```
average_20 = yields['10 Yr'].rolling(20).mean()
rolling_std = changes['10 Yr'].rolling(20).std()
# each value is stored on the last date of its window
```

Standard deviation of the 10 Yr daily change, in bp

5.30
over the whole span

14.51
highest, window ending 2020-03-26

1.93
lowest, window ending 2021-01-05

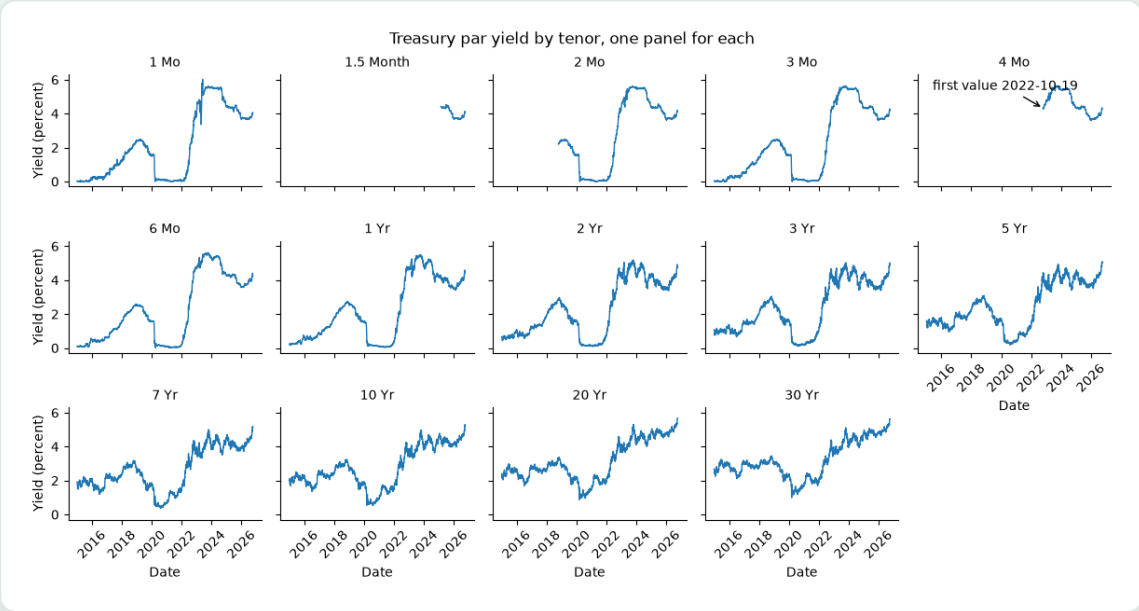
20-row average of the 10 Yr	Percent	Date
First value: the 20th row	1.88	2015-01-30
Lowest	0.59	2020-08-10
Highest	5.05	2026-10-02

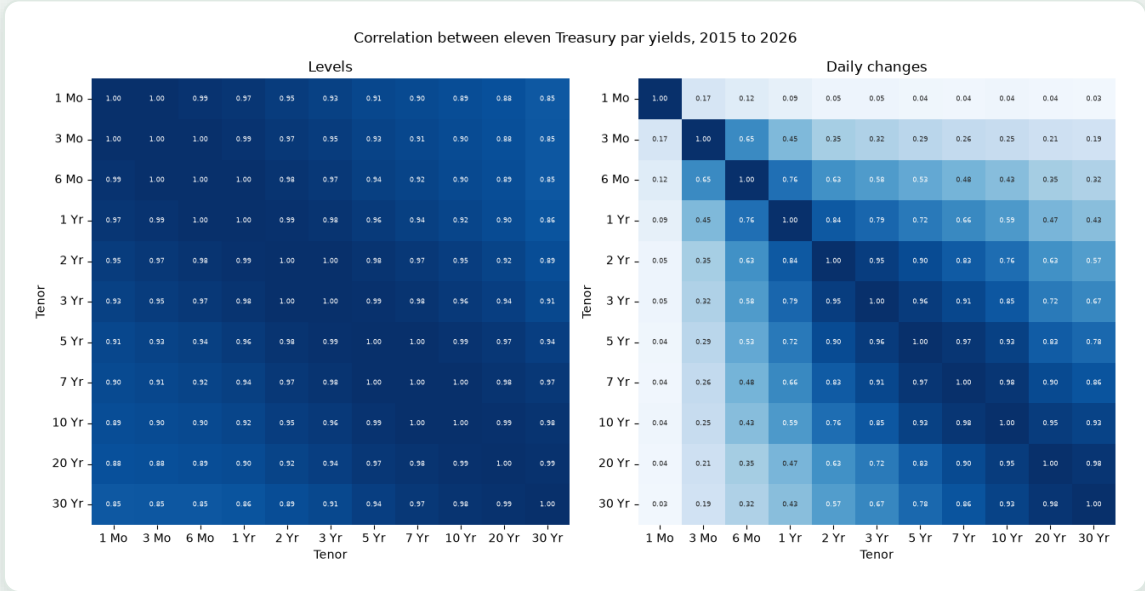
A window is 20 rows, not 20 calendar days. The first 19 dates have no average. Each value describes the 20 rows behind its date. It is not a forecast, and the file gives no reasons.

The same in Excel

`=AVERAGE(M2:M21)`	in S21, filled down to S2941
`=STDEV.S(R3:R22)`	in T22, filled down to T2941
`=MAX(T22:T2941)`	returns 14.51

Every tenor on a panel, and do they move together?






```
sns.relplot(data=long, x='date', y='yield_pct',
            col='tenor', col_wrap=5, kind='line')
```

As levels every pair correlates between 0.85 and 1.00, because all the rates rose and fell together over the years. As daily changes the 7 Yr and 10 Yr are 0.98, the 2 Yr and 10 Yr 0.76, and the 1 Mo and 30 Yr 0.03.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Load a file as a time series, with the dates as the index
- Select a date, a month, a year, or a range of dates with .loc
- Find absent days and blank cells, and say what they do to an average
- Describe yields by tenor and by year, as tables, a heatmap, and line plots
- Work out a spread, daily changes, and a rolling window, with their dates

Week 3, Day 5 Case Study: A Trade Blotter. A short guided EDA on a small synthetic trade blotter